DEVOPS TOPIC OVERVIEW: SECRETS MANAGEMENT

ELIZABETH RONAN AND GRACE KLUENDER

INTRODUCTION AND OVERVIEW

Secrets Management

The secure handling of sensitive information used by applications, services, and infrastructure

What are Secrets?

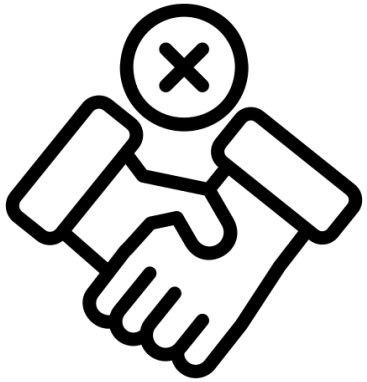
Database credentials, API keys, OAuth tokens, TLS certificates, Cloud access credentials, Encryption keys, SSH keys

WHY DOES THIS MATTER?

Without proper secret management practices, organizations open themselves to a range of threats

Compromised Systems

Data Breaches



Regulatory Violations

Loss of Client Trust

WHY DOES THIS MATTER?

Secrets play a role at every stage of the DevOps lifecycle

Build

- Authenticate to a private repo
- Retrieve artifacts
- Sign code

Deployment

- Inject env configuration into deployed services
- Authenticate infrastructure tooling
- Connect workflows to external services

Runtime

- Continuous authentication
- Service-to-service auth in microservices
- Secret rotation and dynamic credentials

BRIEF HISTORY/EVOLUTION

Pre-DevOps (1990s – 2008ish)

- Secrets were hardcoded into source code or config files
- Credentials were manually shared via email, tickets, or chat
- Long-lived passwords were common
- Shared admin passwords
- Little to no rotation or audit logging

BRIEF HISTORY/EVOLUTION

DevOps Era (2008 – 2015ish)

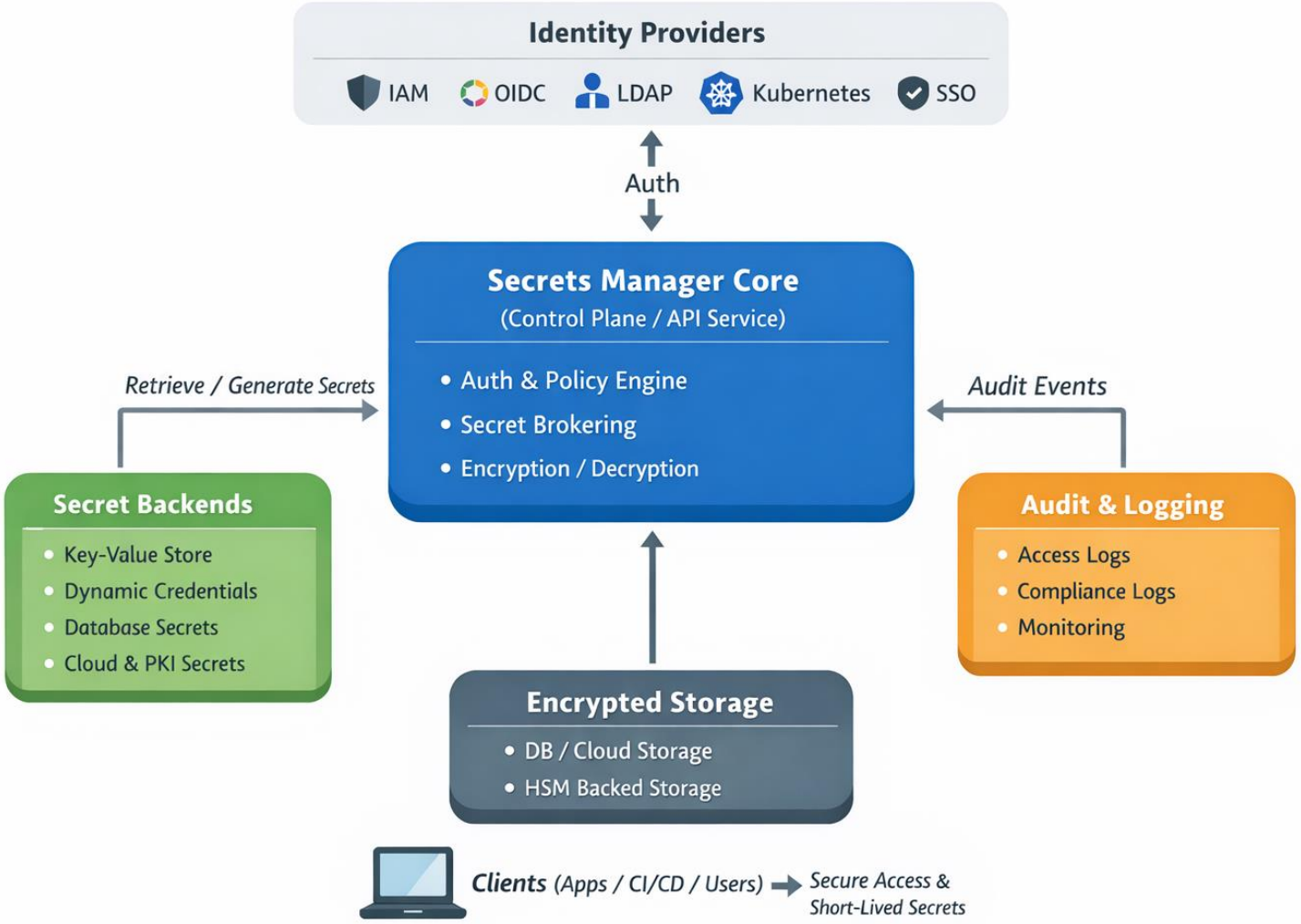
- CI/CD pipelines, containers, microservices, IaC, and cloud services increased the number of secrets exponentially, resulting in "secrets sprawl"
- Instead of pulling secrets from the source code, they are pulled from environmental variables
- IAM roles replaced static credentials in workflows
- Centralized key management services enabled encryption at rest and separated key management from applications

BRIEF HISTORY/EVOLUTION

DevSecOps Era (2015 - Present)

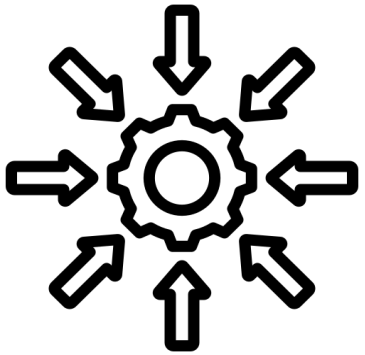
- Dedicated secrets managers became standard (e.g., HashiCorp Vault, AWS Secrets Managers, etc.)
- Secret scanning integrated into CI pipelines
- Audit logging and compliance integration
- Least privilege access enforced through identity-based policies
- Dynamic, short-lived credentials with automatic rotation and expirations

ARCHITECTURE & WORKFLOW



KEY PRINCIPLES & METHODOLOGIES

1. Centralization
2. Identity-based, least-privilege access control
3. Dynamic secrets & short-lived credentials
4. Secret lifecycle management
5. Auditability & compliance
6. Pluggable/Integration-Driven Architecture
7. Defense in depth
8. Secret sprawl prevention



TOOLS AND TECHNOLOGIES

- HashiCorp Vault
- Akeyless Vault Platform
- aws Secrets Manager
- Doppler
- Azure Key Vault
- 1Password Secrets Automation
- Google Secret Manager
- Bitwarden Secrets Manager
- CyberArk® Conjur
- Delinea Secret Server

HASHICORP VAULT

- HashiCorp vault provides centralized, well audited, encryption as a service privilege access and secrets management tool
- Consists of a modular design with building blocks called plugins and a growing plugin ecosystem
- Lets users integrate with current systems to automate application workflow
- Lets you deploy systems in cloud or on-premises environments
- Supports automated secret rotation as well as dynamic secrets (unique, time bound credentials that automatically expire or rotate)

HASHICORP VAULT ALLOWS YOU TO:

- Manage static secrets
- Manage certificates
- Manage identities and authentication
- Create and manage third-party secrets
- Support regulatory compliance

AWS SECRETS MANAGER

- Amazon Web Service's Secrets Manager allows users to manage access to applications, services, and IT resources
- Handles secure storage, rotation, and retrieval of secrets
- Encrypts keys using Amazon Key Management Services, allows users to define access permissions
- Facilitates automated rotation of secrets to enhance security and multi-region replication for high availability

USE CASES

- Centrally store and manage credentials, API keys, and other secrets
- Use AWS Identity and Access Management (IAM) permissions policies to manage access to your secrets
- Rotate AWS and third-party secrets automatically on demand or on a schedule, without redeploying or disrupting active applications
- Integrate secrets with AWS logging, monitoring, and notification services

WHICH TOOL SHOULD I CHOOSE?

HashiCorp Vault

- You require a highly customizable secrets management solution
- You need a service that can run on on-premise environments or in multi-cloud environments
- You're okay with more involved initial setup and maintenance

AWS Secrets Manager

- You already use AWS as a cloud provider and are invested in the AWS ecosystem
- You are a young company with relatively simple infrastructure looking for a fully managed secrets management service
- You're looking for simple deployment and don't require much customization

TECHNICAL DEMO

CREATE SECRET FOR RDS DATABASE

Secrets | Secrets Manager | u
database-ge - Database Deta
Create user | IAM | Global
AdministratorAccess | IAM | G
Gemini

us-east-1.console.aws.amazon.com/secretsmanager/listsecrets?region=us-east-1
Relaunch to update

aws
Search [Option+S]
United States (N. Virginia)
eronan22 (8349-2293-4112)
eronan22

AWS Secrets Manager > Secrets

Secrets

Store a new secret

Filter secrets by name, description, tag key, tag value, owning service or primary Region

1

Secret name	Description	Last retrieved (UTC)	Created on (UTC)
rds!db-9e259e04-e679-4537-b0f5-70289dc2fde7	The secret associated with the primary RDS DB instance: arn:aws:rds:us-east-1:834922934112:db:database-ge	-	February 25, 2026 at 02:03:51

CloudShell Feedback Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy Terms Cookie preferences

CREATE USER AND IAM POLICY

demo_secret | Secrets Manag | database-ge - Database Deta | Users | IAM | Global | AdministratorAccess | IAM | G | AdministratorAccess | IAM | G | Gemini

us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/users

Relaunch to update

aws | Search [Option+S] | Global | eronan22 (8349-2293-4112) | eronan22

IAM > Users

Identity and Access Management (IAM)

Search IAM

- Dashboard
- **Access Management**
  - User groups
  - Users
  - Roles
  - Policies
  - Identity providers
  - Account settings
  - Root access management
  - Temporary delegation requests New
- **Access reports**
  - Access Analyzer
    - Resource analysis New
    - Unused access
    - Analyzer settings
  - Credential report
  - Organization activity
  - Service control policies

Users (1) Info

Delete | Create user

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Search

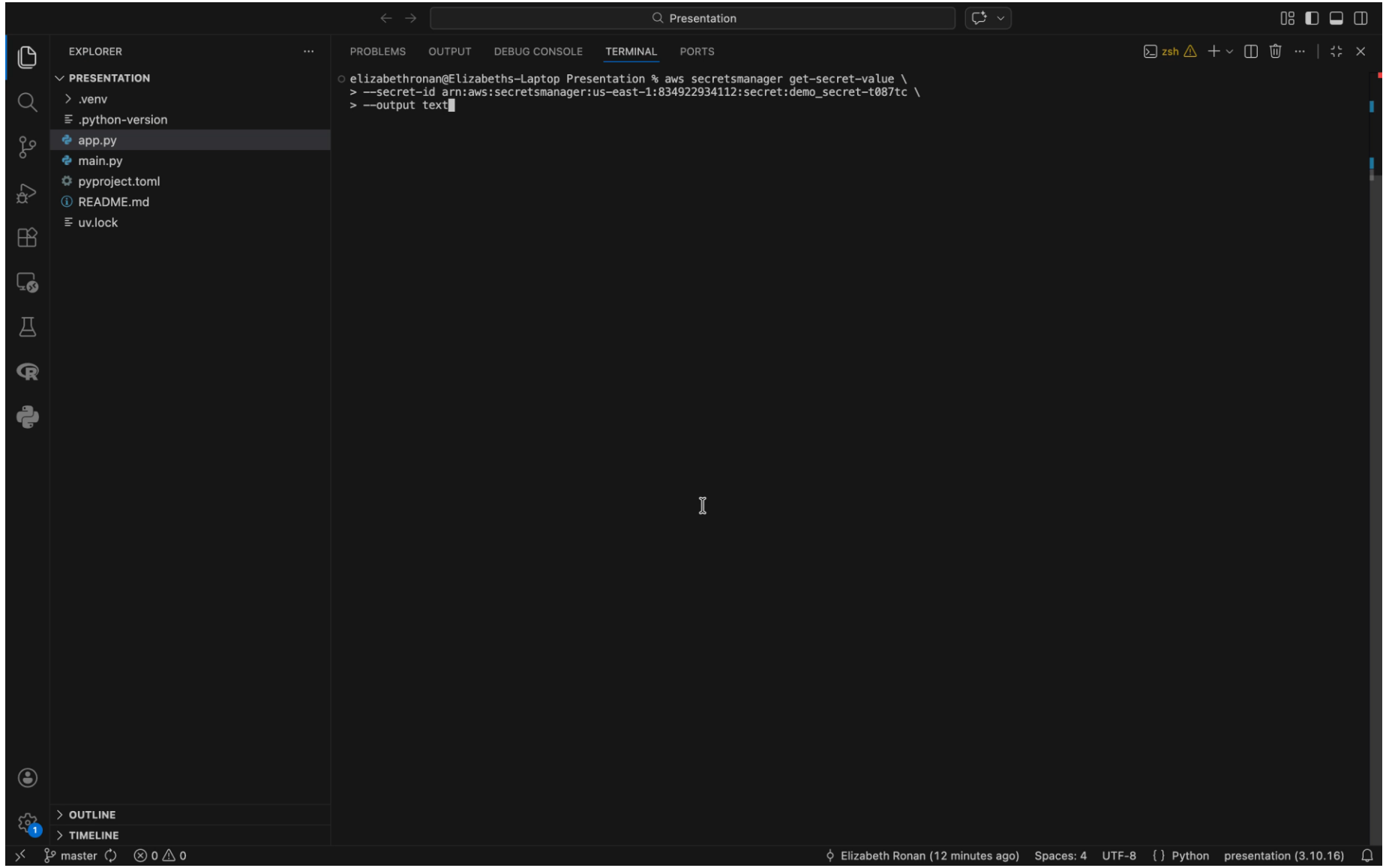
1

User name	Path	Groups	Last activity	MFA	Password age	Console last sign-in
demo-user	/	0	-	-	9 minutes	-

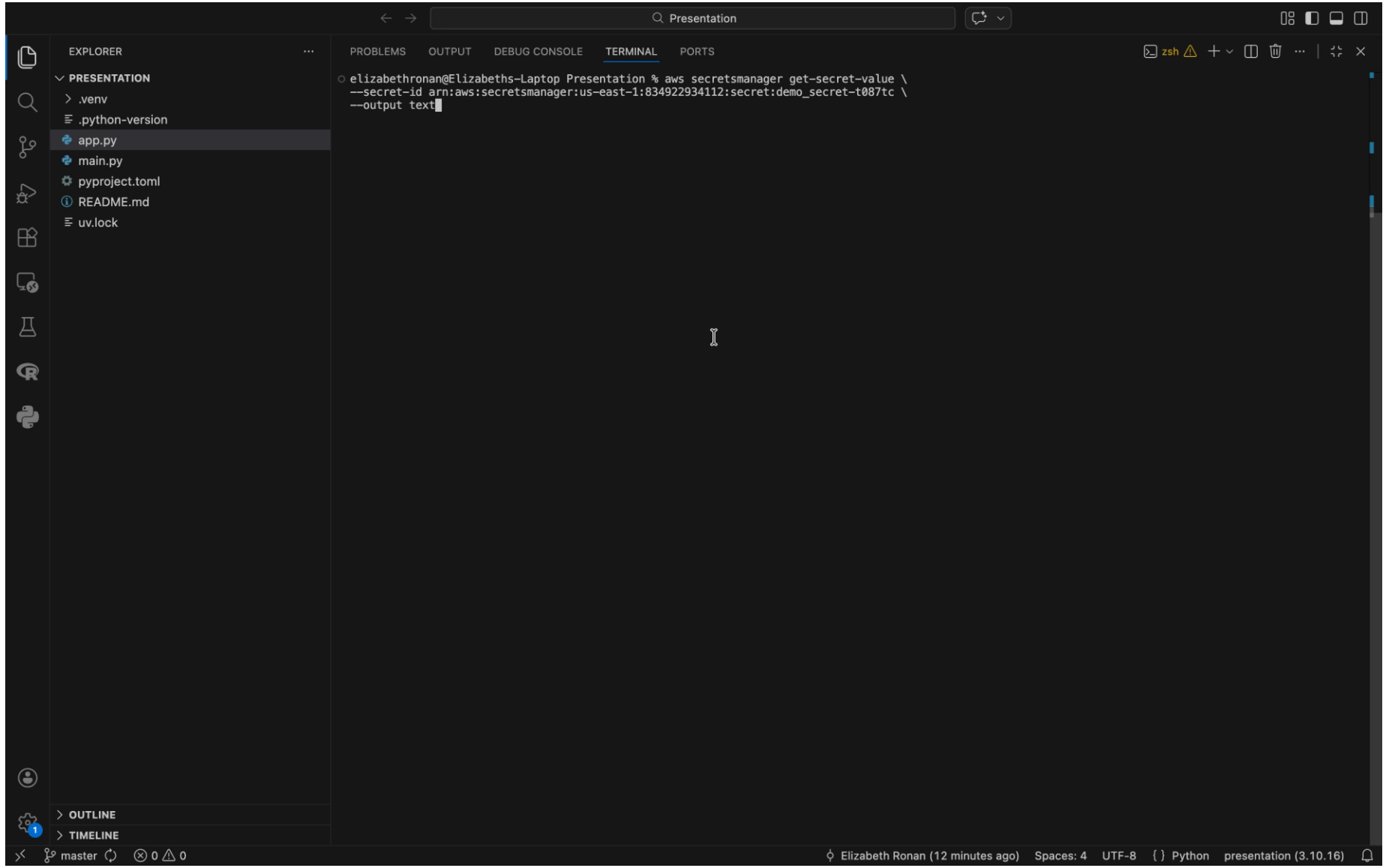
CloudShell | Feedback | Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy | Terms | Cookie preferences

ADMINISTRATOR SECRET RETRIEVAL



UNAUTHORIZED SECRET RETRIEVAL



IN CODE IMPLEMENTATION: CONNECT TO DATABASE WITH SECRET

```python
import psycopg2
import boto3
import json, subprocess

def get_secret():
    """
    Extracts the requested secret from AWS. Only works if configured user has proper permissions through its IAM role.
    """
    try:
        secret = subprocess.check_output(
            ['bash',
            '-lc',
            "aws secretsmanager get-secret-value --secret-id 'arn:aws:secretsmanager:us-east-1:834922934112:secret:rds!db-9e259e
            ]
        ).decode(); password = json.loads(secret)['password']
        print('Secret retrieved!')
    except subprocess.CalledProcessError as e:
        print(f'Subprocess error: {e}')

    return secret, password
```

```python
def get_conn(password):
    """
    Connect to example Amazon RDS database using password retrieved from secret.
    If connection is successful, print success message and close.
    """
    conn = None
    try:
        conn = psycopg2.connect(
            host='database-ge.cspks6qqgm4n.us-east-1.rds.amazonaws.com',
            port=5432,
            database='postgres',
            user='postgres',
            password=password,
            sslmode='verify-full',
        sslrootcert='/Users/elizabethronan/global-bundle.pem'
        )
        cur = conn.cursor()
        print('Database connection successful!')
        cur.execute('SELECT version();')
        print(f'Database version: {cur.fetchone()[0]}')
        print('Closing connection.')
        cur.close()
    except Exception as e:
        print(f"Database error: {e}")
        raise
    finally:
        if conn:
            print('Closing connection.')
            conn.close()
```

REAL WORLD IMPLEMENTATION: BENEFITS

Avoid security islands: prevents instances of only a small group of users having specialized access

Identify bad actors: strong verification requirements cuts off hackers attempts to appear as a credentialed user

Automating credentials: reduces the amount of manual needed to authenticate users and allow workflows

REAL WORLD IMPLEMENTATION: CHALLENGES

- Secret sprawl: secrets are scattered across systems in ways that make it challenging to protect them all
- Default behaviors sometimes store secrets in insecure places, for example, embedded in plain text configuration files
- Even non-public password storage can open organizations up to malicious insider attacks

BEST PRACTICES

1. Centralize secret storage
2. Implement least privilege access
3. Automate rotation
4. Remove hard coded secrets
5. Enable comprehensive auditing
6. Encrypt everything
7. Use dynamic secrets

FUTURE TRENDS

1. AI-Driven Secrets Automation
   - AI can detect abnormal secret usage, predict leaks, and automatically trigger rotation and revocation.
2. Autonomous / Self-Healing Secrets Lifecycle
   - When security events are detected, secrets are rotated or revoked without manual intervention.
3. Secretless Architectures
   - Identity replaces credentials, removing secrets entirely, and relying on continuous identity verification.

SOURCES

- https://www.redhat.com/en/topics/devops/what-is-secrets-management
- https://jeevisoft.com/blogs/2025/06/what-is-secret-management-in-devops-and-why-it-matters/
- https://devsecopsschool.com/blog/secrets-management-in-devsecops-a-comprehensive-guide/
- https://pushbuildtestdeploy.com/why-do-we-need-secrets-managers/
- https://www.geeksforgeeks.org/devops/aws-secrets-manager/
- https://developer.hashicorp.com/vault/docs/plugins
- https://infisical.com/blog/aws-secrets-anager-vs-hashicorp-vault
- https://www.wallarm.com/cloud-native-products-101/vault-vs-aws-secrets-manager-secrets-management
- https://www.redhat.com/en/topics/devops/what-is-secrets-management
- https://www.aquasec.com/cloud-native-academy/supply-chain-security/secrets-management/
- https://managesecrets.dev/article/The_future_of_secrets_management_and_emerging_trends.html
- https://www.techaheadcorp.com/blog/key-devsecops-trends/
- https://phoenixnap.com/blog/devops-trends